UNIVERSITÀ
DEGLI STUDI
DELL'AQUILA

Lesson 8: Unsupervised Learning

Technologies for Artificial Intelligence

Fabio Antonini, Università degli Studi dell'Aquila

13 November 2026

Agenda

- A fourth question this course asks of data: no target at all
- k-means: the objective, Lloyd’s algorithm, choosing k
- Hierarchical clustering and DBSCAN: shape versus density
- Validating a clustering with no labels
- Principal component analysis (PCA) and t-SNE

Exercise 7, before we start

- What counts is the **defence of the chosen family**, not the winning score
- The gap to watch for: a feature-importance ranking quoted without checking how many candidate columns competed for attention at each split
- Today’s methods come with a version of the same obligation: a clustering or a projection is a claim, and it needs its own check

No y at all

- Every method so far learned from pairs: input x, target y
- **Unsupervised learning** removes y: measurements, no answer key
- The question: not “predict the label” but “describe the structure”
- Most organisational data was never labelled: expensive, slow, often not well-defined
- Often the *first* pass on a dataset

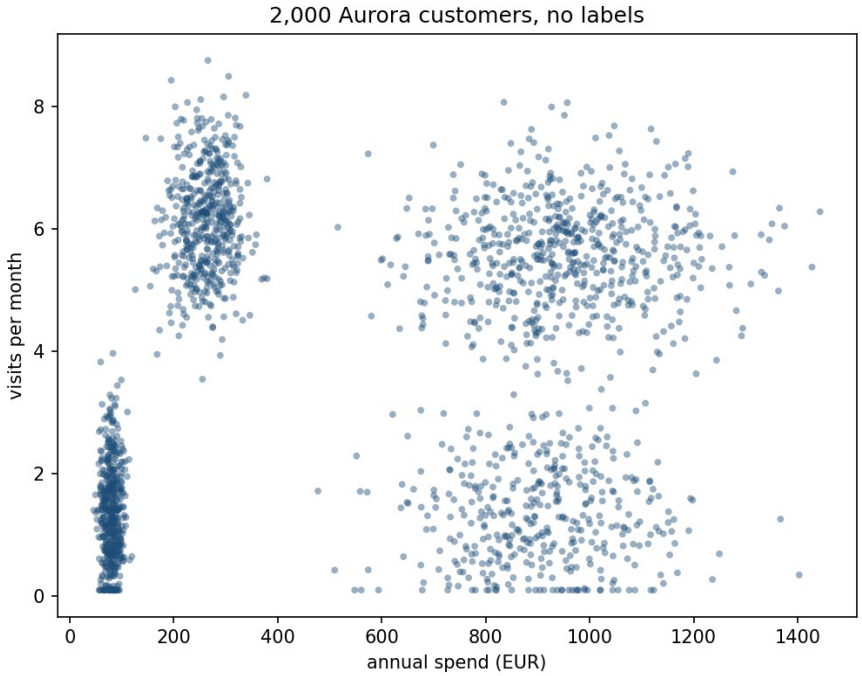
Aurora: three problems, no answer key

- **Segment 2,000 customers**, from spend and visits (**k-means**, nb. 01)
- **Tell bots from humans** in 1,500 sessions, same *average* behaviour (**hierarchical clustering, DBSCAN**, nb. 02)
- **Catch disguised fraud** in an 8-column table, ordinary on every column (**PCA**, nb. 03)
- Every dataset is synthetic: its truth is normally hidden

A ground truth Aurora will never have

- `Notebooks/retail_data.py` publishes the **true** structure as `TRUE_*` constants
- This lesson can check every method against a real answer: Aurora’s own analysts never could
- Hold onto that asymmetry: it makes today teachable, and it is exactly what is missing the first time this runs on real data

2,000 customers, no labels



Turning “alike” into a number

- A good grouping puts every point close to one representative “typical customer,” and far from every other group’s
- **k-means** turns “close” into a number and minimises the total
- Two things are searched for jointly: every point’s **assignment**, and the k cluster **centres**
- Both at once is hard; the trick is minimising each separately, holding the other fixed

The objective: within-cluster sum of squares

- Minimise, jointly, over the cluster centres μ_j **and** every assignment r_{ij}
- $r_{ij} = 1$ if point i is assigned to cluster j, else 0: exactly one 1 per row
- This quantity has a name: **within-cluster sum of squares (WCSS)**, or *inertia* in scikit-learn

Minimise J

$$J = \sum_{i=1}^{m} \sum_{j=1}^{k} r_{ij} \, \|x_i - \mu_j\|^2$$

Lloyd’s algorithm: assign, then update, repeat

- **Assign step**: centres fixed, send every point to its **nearest** centre: exact minimiser over assignments
- **Update step**: assignment fixed, move every centre to the **mean** of its points, where the algorithm’s name comes from
- Alternate the two until nothing changes
- Each step is an exact minimisation, no approximation, no step size

Every step can only lower J

- Assign and update each solve an exact minimisation, so neither step can ever **increase** J
- J is bounded below by 0, with only finitely many ways to partition m points into k groups
- Never increasing, cannot fall forever: the sequence must repeat, and that is convergence
- It converges to a **local** minimum, not necessarily the **global** one

Worked example: six customers, badly-chosen starts

Iteration	Assignment	WCSS after assign	WCSS after update
0	`[0 1 1 1 0 1]`	15.965	7.281
1	`[0 0 1 1 0 1]`	5.491	4.674
2	`[0 0 1 1 0 1]`	4.674	4.674

A bad start is a real risk

- 30 independent naive random starts of Lloyd’s algorithm on the 2,000-customer data
- Best final WCSS: **374.1**. Worst: **1,390.4**, **3.7 times worse**, same data, same update rule
- This happened on the very **first** attempt (worst of 30), not a rare one-in-a-thousand pathology
- The naive fix (k points picked uniformly at random) gives no way to tell in advance which kind of run you got

k-means++: favour centres far from what's chosen

$$P(x_i \text{ chosen next}) = \frac{d(x_i)^2}{\sum_{i'} d(x_{i'})^2}, \qquad d(x_i) = \min_{j \text{ chosen}} \|x_i - \mu_j\|$$

k-means++, measured

- Same 30 seeds, k-means++ instead of naive random
- Range narrows to **374.1–1,088.3**: the worst case gains ~300 units
- Reaches the **global optimum on the first attempt**
- `sklearn.cluster.KMeans` uses k-means++ plus 10 restarts by default
- Notebook 01 matches its WCSS to 10^{-4}

Choosing k

- k is an **input** to k-means, not something the algorithm discovers
- WCSS is monotonically non-increasing in k: at k=m, J=0 exactly
- So the raw value of J at the “best” k is not the signal
- **The elbow**: where the WCSS curve stops falling steeply, past the true count, an extra centre only subdivides an already-coherent group

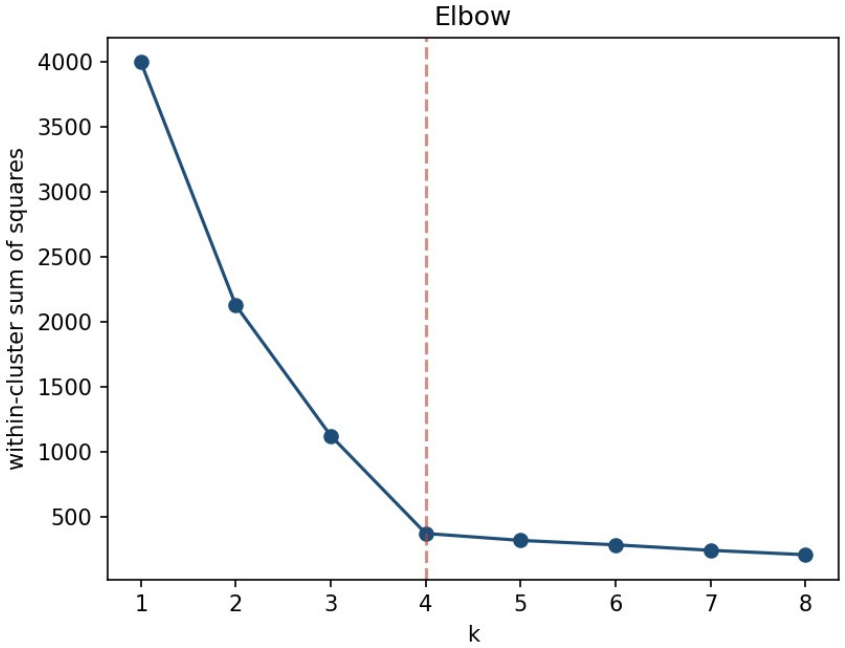
The silhouette score

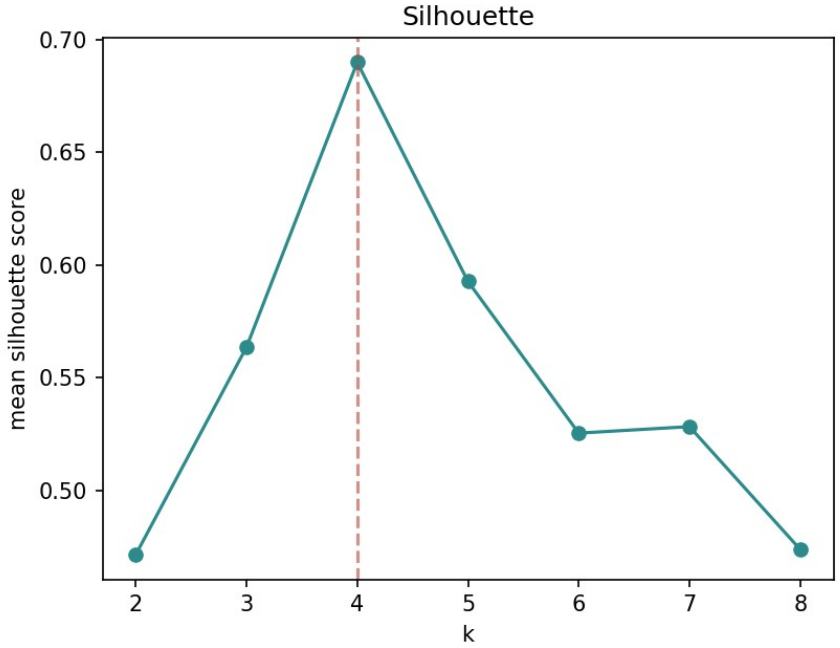
- For point i: a_i is its mean distance to every other point in its **own** cluster
- b_i is its mean distance to the points of the **nearest other** cluster
- Combine the two into a single number per point

Between -1 and 1

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)} \in [-1, 1]$$

Elbow and silhouette, on the same data

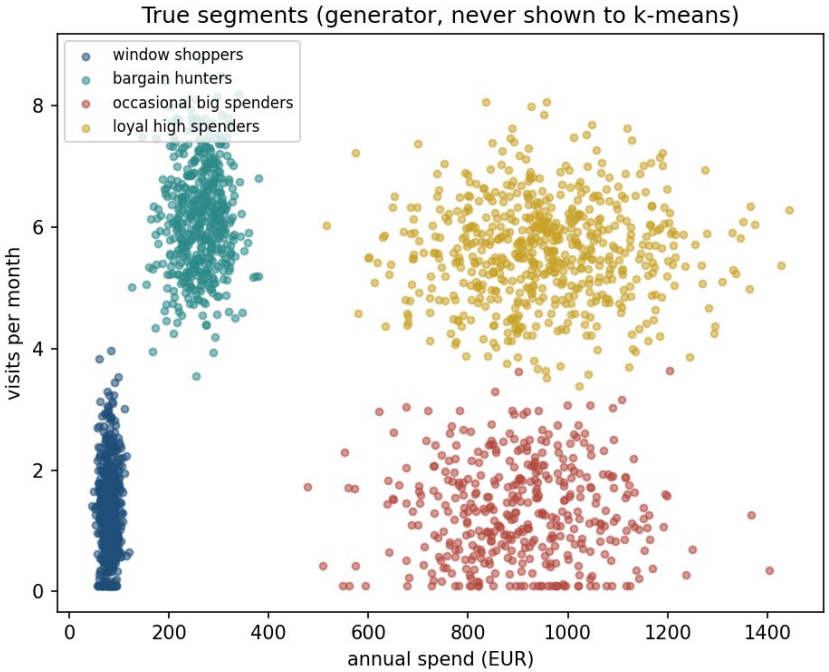


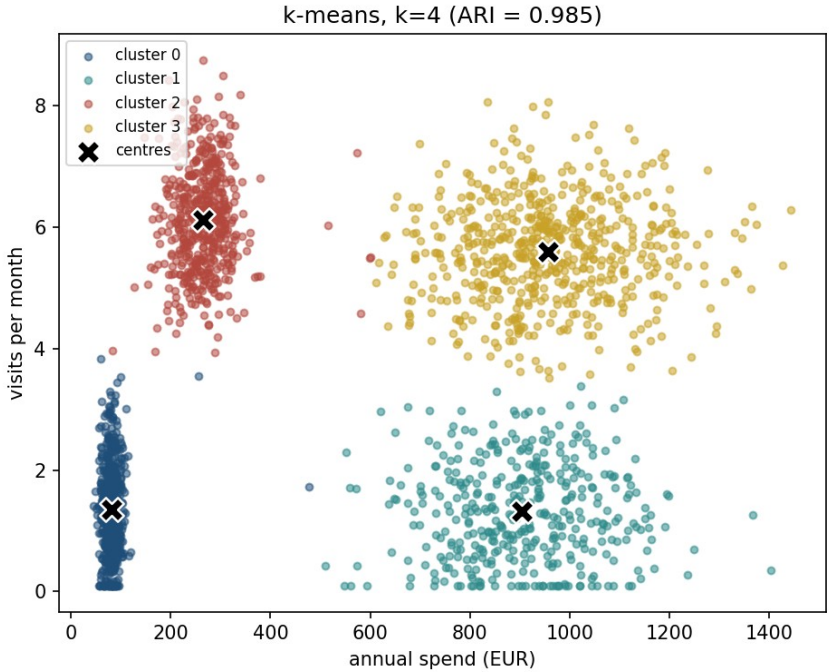


Both point to k=4

- WCSS at k=4: **374.1**, the sharpest bend in the elbow curve
- Silhouette at k=4: **0.690**, the peak across k=1 to 8
- Both match the number of segments the data was actually built with
- Agreement here is worth noting precisely **because** it does not always happen, and is not required to

k-means against a ground truth Aurora doesn’t have





UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The adjusted Rand index

- Compares two labellings: a **random** assignment scores 0, **identical** labellings score 1
- Cluster “0” need not mean the same thing on both sides: relabelling is handled
- k-means vs. the true segments: **0.985**, near-perfect
- An **external** metric: needs a ground truth, which section 4 asks about when there isn’t one

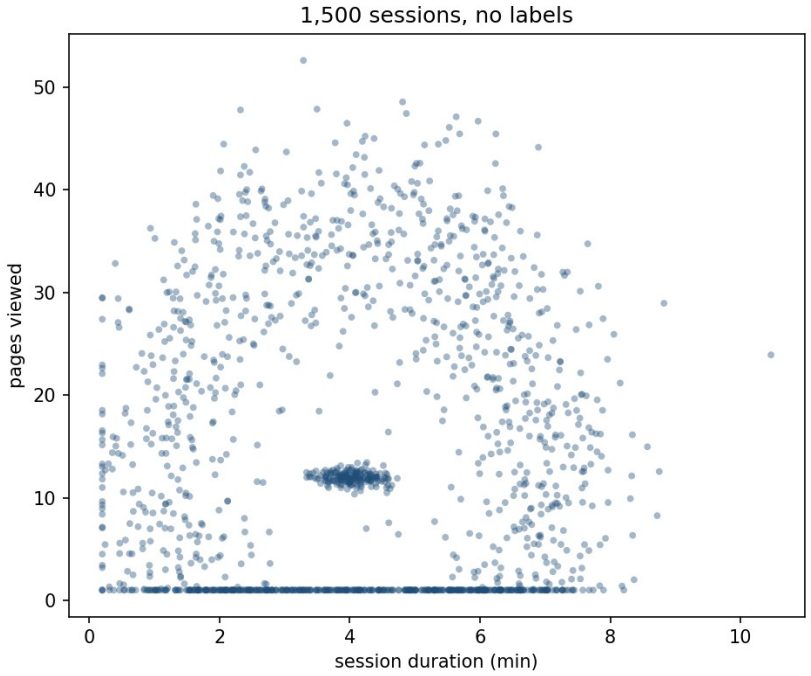
Notebook 1, live

- k-means from scratch, checked against scikit-learn’s k-means++ and 10-restart defaults
- The 30-seed initialisation comparison, reproduced live
- The elbow and silhouette curves, computed directly from the data

A dataset built to defeat k-means

- Security suspects some of 1,500 sessions were automated
- Two features: duration in minutes, pages viewed
- **12%** are, by construction, **bots**: scripted and consistent
- Bots and humans share the **same mean**: a tight cloud inside a diffuse ring
- No straight cut separates them: there are no two offset clouds

1,500 sessions, unlabelled



k-means on this shape

- Run k-means with k=2 on the session data
- ARI: **-0.046**, indistinguishable from a random split
- Not a tuning or initialisation failure, no starting point fixes it
- WCSS wants round, compact clusters; the only cut through this cloud slices **both** the bot core and the human ring in half

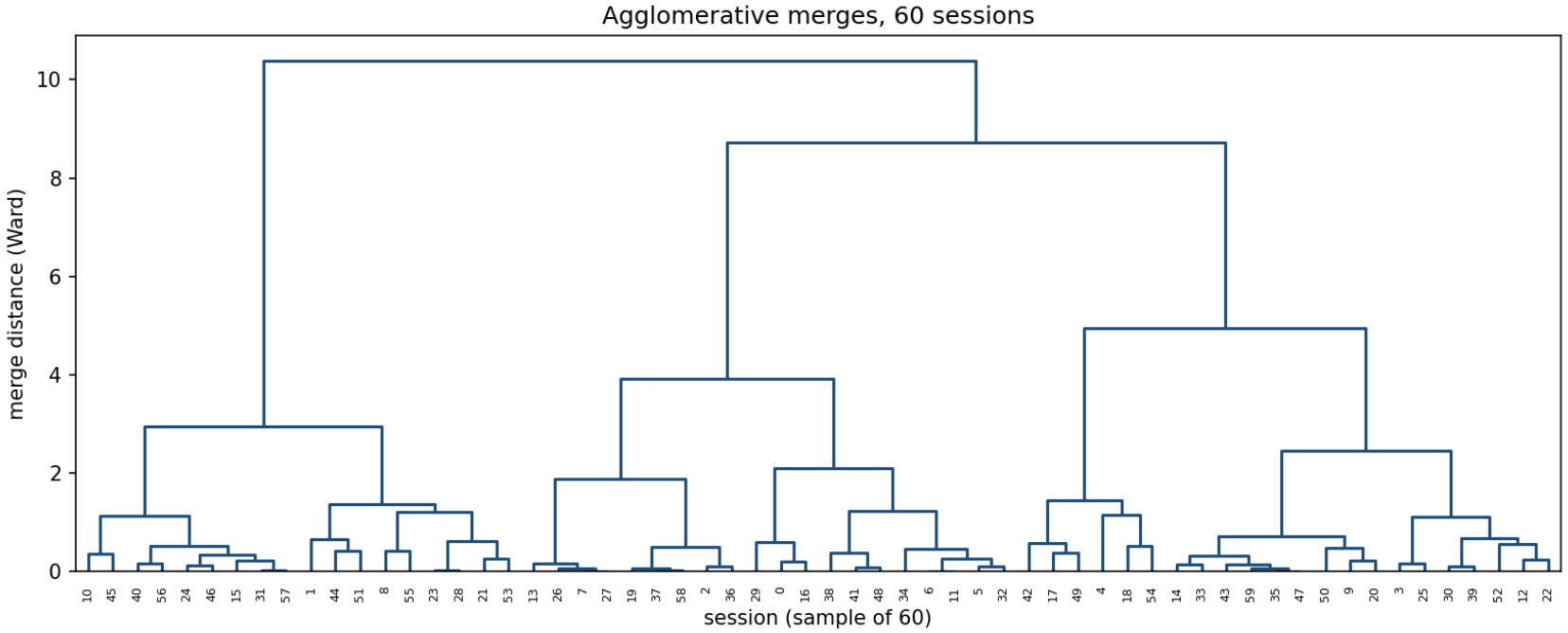
Merge the two closest

- Start with every point as its **own** cluster
- Repeatedly merge the two **closest** clusters, recording every merge as a branch in a tree: a **dendrogram**
- Cutting the tree at a height is equivalent to choosing k: branches crossed = cluster count
- “Closest” needs a rule for distance **between clusters**: the **linkage criterion**

Four notions of “closest”

- **Single**: closest pair, which can chain a winding path, or bridge two groups at one touching point
- **Complete**: farthest pair, which favours compact groups
- **Average**: mean distance over every cross-pair, a compromise
- **Ward**: least added variance, the *same* objective k-means minimises

Ward-linkage merges on 60 sessions



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

All four linkage rules, scored

Linkage	ARI	Cluster sizes
Ward	-0.062	1,007 / 493
Complete	-0.101	1,253 / 247
Average	-0.001	1,499 / 1
Single	-0.001	1,499 / 1

Break

- Twelve minutes

DBSCAN: density instead of shape

- **DBSCAN**: density-based spatial clustering of applications with noise
- Classifies points by neighbourhood crowding: radius `eps`, minimum count `min_samples`
- **Core point**: at least `min_samples` others within `eps`
- **Border point**: within `eps` of a core, but not core itself
- Everything else: **noise**

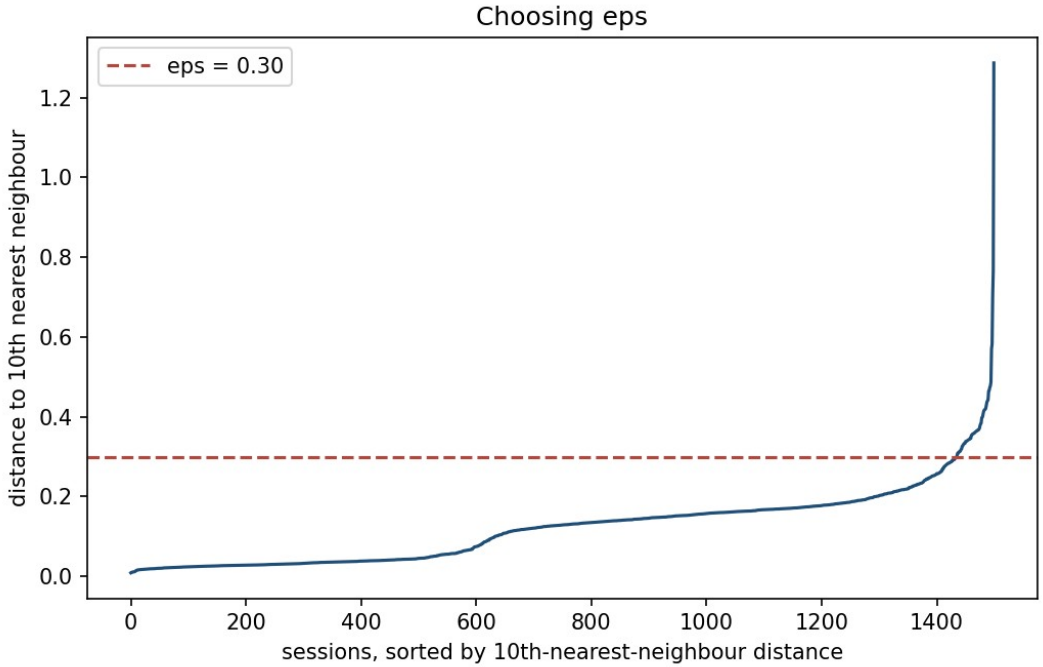
What counts as a cluster

- A **maximal set of core points**, chained by mutual `eps`-closeness, plus every attached border point
- No centre, no mean, no shape assumption in the definition
- A compact core and a diffuse ring both count, if dense enough at the scale `eps` measures
- Points can belong to **nothing**: noise is first-class, not an error

Choosing eps: the parameter that matters most

- Too small: even the dense bot core fractures into isolated noise points
- Too large: the ring bridges back into the core, everything merges, as Ward linkage did
- A **k-distance plot**: distance from every point to its `min_samples`-th nearest neighbour, sorted
- Dense regions give a small distance, sparse ones a large one: the plot typically bends sharply between the two

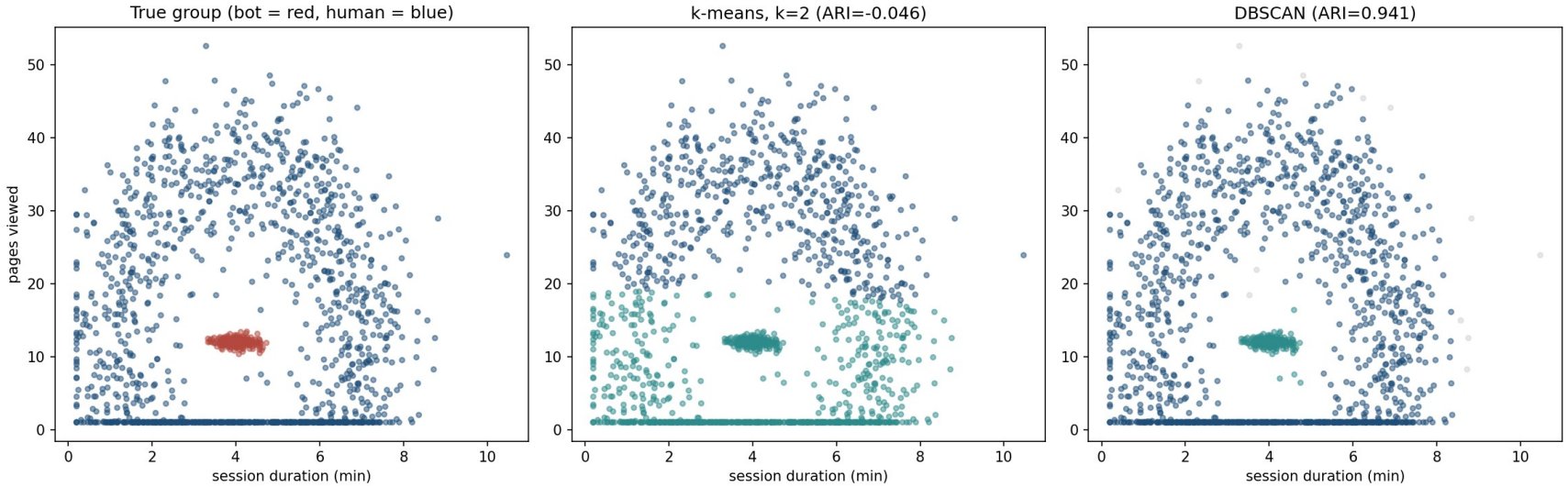
The bend in the curve



DBSCAN, measured

	Result
ARI against true bot/human split	**0.941**
Core points	1,435 of 1,500
Noise points	13

Three views, side by side



Does `eps` transfer?

Week	ARI at `eps = 0.30`	Knee `eps`	ARI at knee
this one	**0.941**	0.258	0.890
week 4	**−0.012**, *one cluster*	0.226	0.858
week 5	**−0.009**, *one cluster*	0.232	0.887
the other four	0.93 – 0.96	0.21 – 0.23	0.79 – 0.85

Choosing among the three

Situation	Reach for
Clusters plausibly round, similar size, k known or searchable	k-means
Want the hierarchy itself: nested groupings at every scale	Agglomerative clustering
Clusters irregular, nested, or of very different densities; outliers should be flagged	DBSCAN
Dataset is very large, every point must be assigned to something	k-means (DBSCAN’s noise needs a policy)

Notebook 2, live

- Agglomerative clustering, all four linkage rules, and the dendrogram
- DBSCAN, the k-distance plot, and the `eps` sweep
- Every result checked against the true bot/human labels

No answer key this time

- Every check so far, ARI in sections 2 and 3, used a ground truth this lesson happens to have because the data is synthetic
- Aurora’s actual analysts, on real customer or session data, will never have that
- Two families of metric fill the gap, and it matters which one is available in a given situation

Internal metrics: only the clustering and the data

- Use **no** outside answer key: computed from the clustering and the points alone
- **Silhouette**: are points closer to their own cluster than to any other? Answerable from the clustering alone
- **WCSS**: usable, but weaker, only comparable across different k on the *same* algorithm, not across different methods
- Always available, even when nothing else is

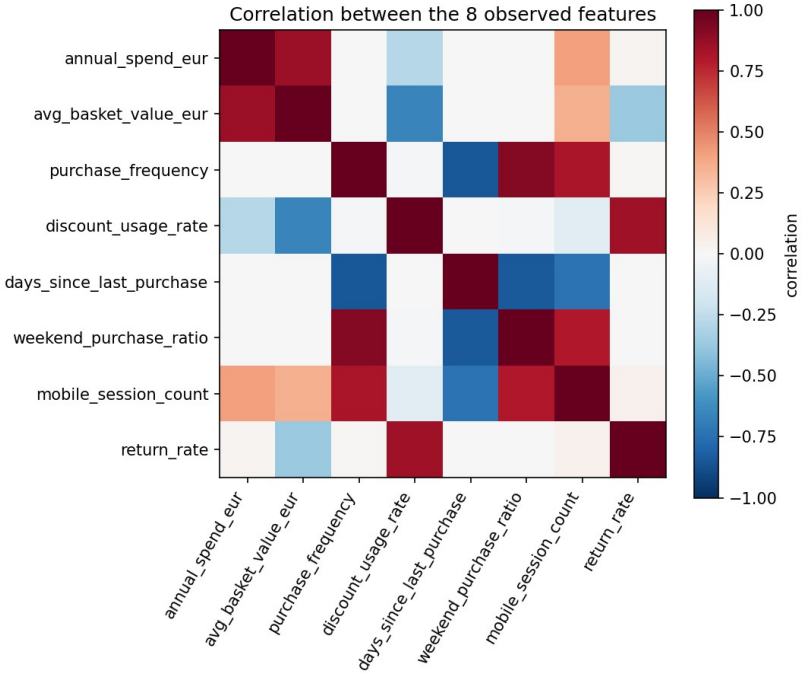
External metrics: an outside answer key

- Compare a clustering against a **separate** labelling assumed correct, ARI and its relatives
- Here: the data generator's `true_segment` / `true_group`
- In practice: a hand-labelled sample, a business rule, or a downstream outcome: did the "bot" cluster get blocked and never return?
- When such a signal exists, even partial or noisy, **use it**

What silhouette can’t see

- A silhouette of 0.69, reported alone, sounds like strong endorsement
- It endorses only **what the metric can see**: the shape, not the assumption behind it
- k-means on the bot/human data (ARI ≈ -0.046) still scores positive
- Silhouette measures **roundness**, not correctness, and k-means always looks round by construction

Eight columns, not eight independent numbers



PCA: the direction that varies most

- Finds the direction in feature space along which the data varies **most**: the first principal component
- Then the next-most-varying direction, **perpendicular** to the first, and so on
- “Varies the most” stands in for “carries the most information”: a direction where every point looks nearly identical tells points apart hardly at all
- A direction like that is safe to discard

Maximise variance, subject to unit length

- The variance of standardised data projected onto a direction v, with $\|v\| = 1$, is what PCA maximises
- Σ is the sample covariance matrix: diagonal 1, off-diagonal entries the correlations the previous figure showed as colours
- The unit-length constraint matters: without it, variance grows without bound simply by scaling v up

Variance along a direction

$$\mathrm{Var}(Xv) = v^\top \Sigma v, \qquad \Sigma = \frac{1}{m-1} X^\top X$$

The stationary points are eigenvectors of Σ

$$\Sigma v = \lambda v$$

The eigenvalue is the variance that direction captures

- Substituting $\Sigma v = \lambda v$ back: $v^T \Sigma v = \lambda$
- The direction of **largest** variance is the eigenvector with the **largest** eigenvalue; the second is next-largest, orthogonal to the first
- Orthogonality is automatic: Σ is symmetric, by the spectral theorem
- Order all n eigenvalues largest to smallest: every component, at once

Worked example: spend and visits

	Value
Correlation (off-diagonal of Σ)	0.171
Eigenvalues	1.171 and 0.829
First component direction	(1,1)/√2

The same answer, more stably: the SVD

- The **singular value decomposition (SVD)** of the centred data is the numerically preferred route to the same components
- Squaring a matrix squares its condition number, never form $X^{T}X$ explicitly

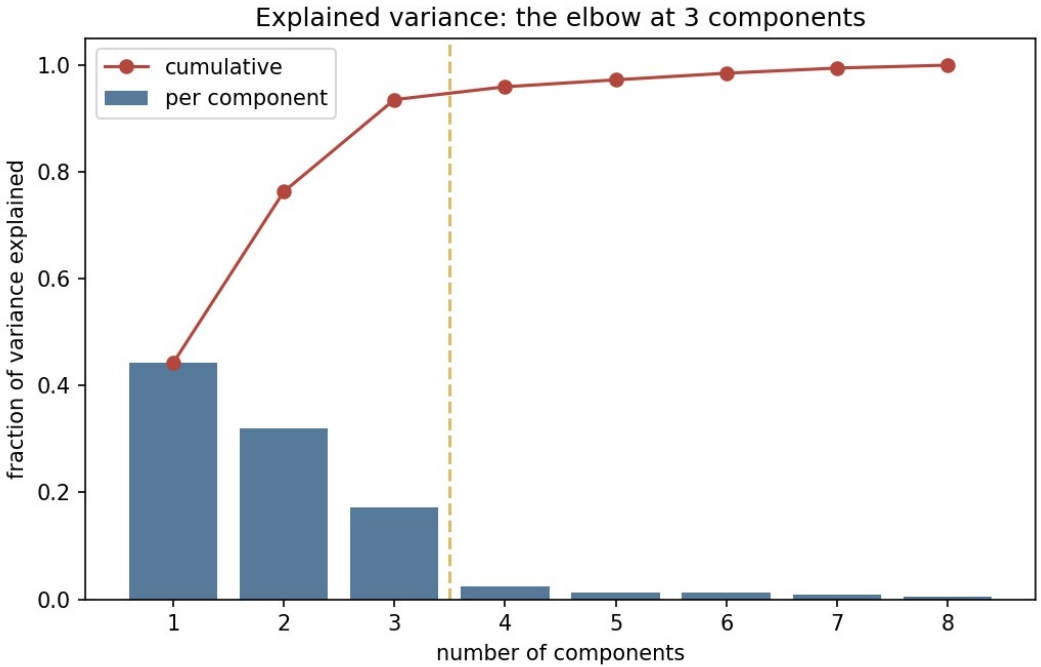
λ_i from singular values

$$X = USV^\top, \qquad \lambda_i = \frac{s_i^2}{m-1}$$

Three routes, one answer

- `numpy.linalg.eigh` on the covariance matrix
- `numpy.linalg.svd` on the centred data, with $\lambda_i = s_i^2/(m-1)$
- `sklearn.decomposition.PCA`
- On the 8-feature account table, all three agree to within 5×10^{-15}: floating-point rounding, and nothing more

How many components: the scree plot



The elbow lands on 3

Component	Eigenvalue	Variance	Cumulative
PC1	3.539	44.2%	44.2%
PC2	2.568	32.1%	76.3%
PC3	1.380	17.2%	93.5%
PC4	0.192	2.4%	96.0%
PC5–PC8	0.043–0.104	0.5–1.3% each	100.0%

What to do when the elbow is softer

- This dataset’s elbow is unusually clean; most real data is not
- Common alternative: keep enough components for a chosen fraction of the variance, typically 90–95%
- Or treat the number of components as a hyperparameter, and let cross-validation decide
- The scree plot starts the judgement; it does not automate it

Notebook 3, live

- PCA on the 8-feature account table, checked three ways, to 5×10^{-15}
- Reconstruction error flags disguised fraud: **37 of 40** planted anomalies caught at the 98th-percentile threshold
- A 2-D PCA and t-SNE scatter of the same accounts, and why neither shows the anomalies as outliers

t-SNE: a tool built for looking, not for measuring

- **t-SNE** (t-distributed stochastic neighbour embedding) arranges points so those close together **stay** close
- Distances between points that started far apart are not preserved
- Excellent at making separate clusters visually obvious, more so than PCA’s first two components
- **Distances between clusters mean nothing**

What to take away

- k-means converges only to a **local** minimum: 30 starts ranged WCSS 374.1 to 1,390.4
- A core inside a ring defeats k-means **and** Ward; DBSCAN scored **0.941**
- Internal metrics measure self-consistency, external (ARI) a truth real problems rarely have
- **37 of 40 disguised accounts** caught by reconstruction error, invisible to a 2-D scatter

Homework

- **Exercise 8**, discussed at the start of **Friday 20 November**
- `Exercises/08_unsupervised_learning.md`